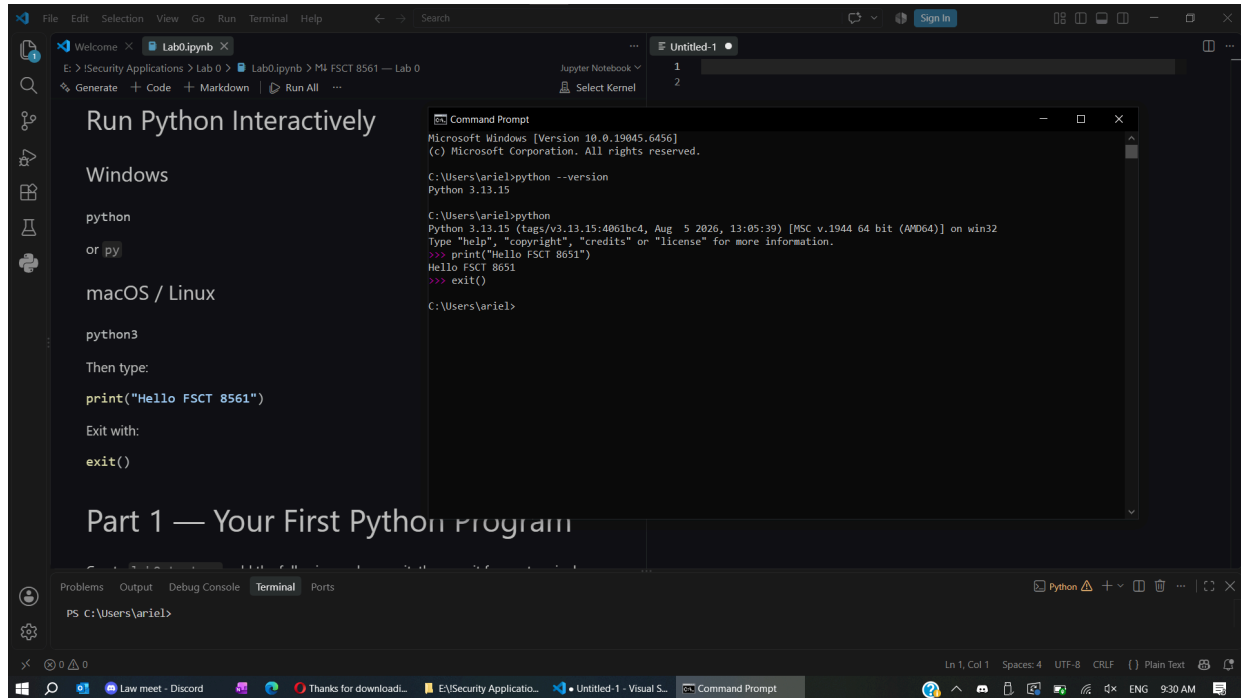


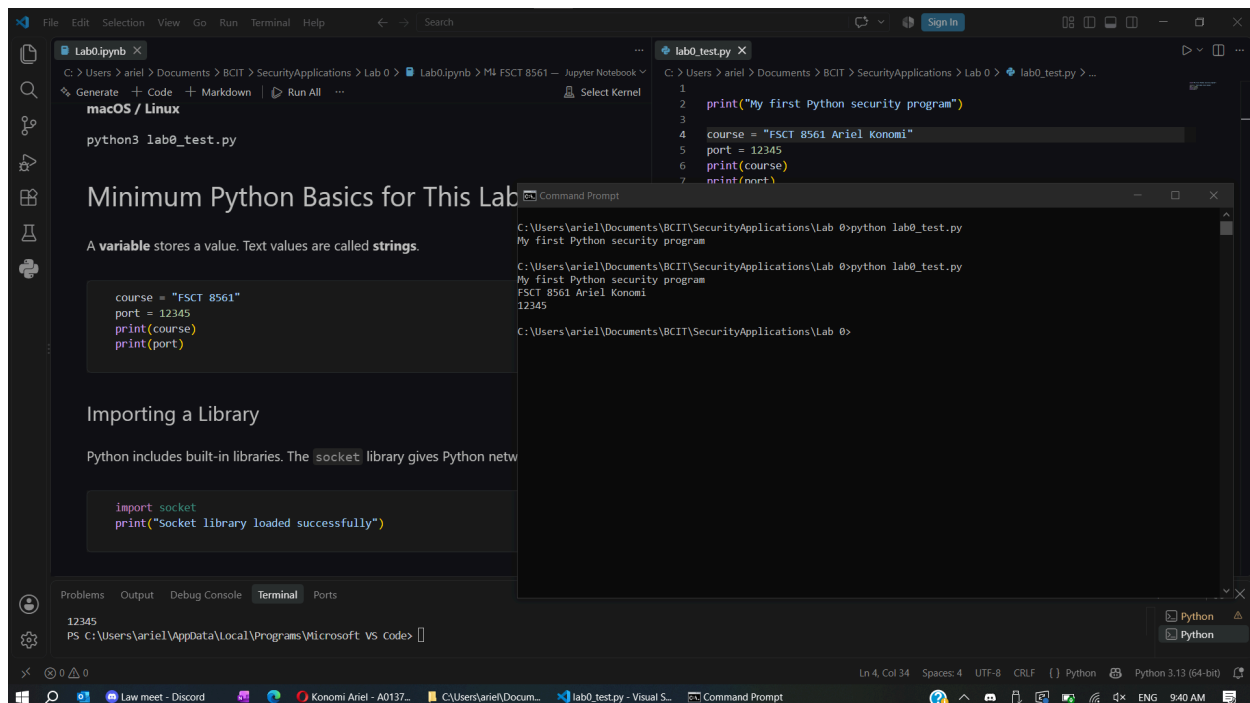
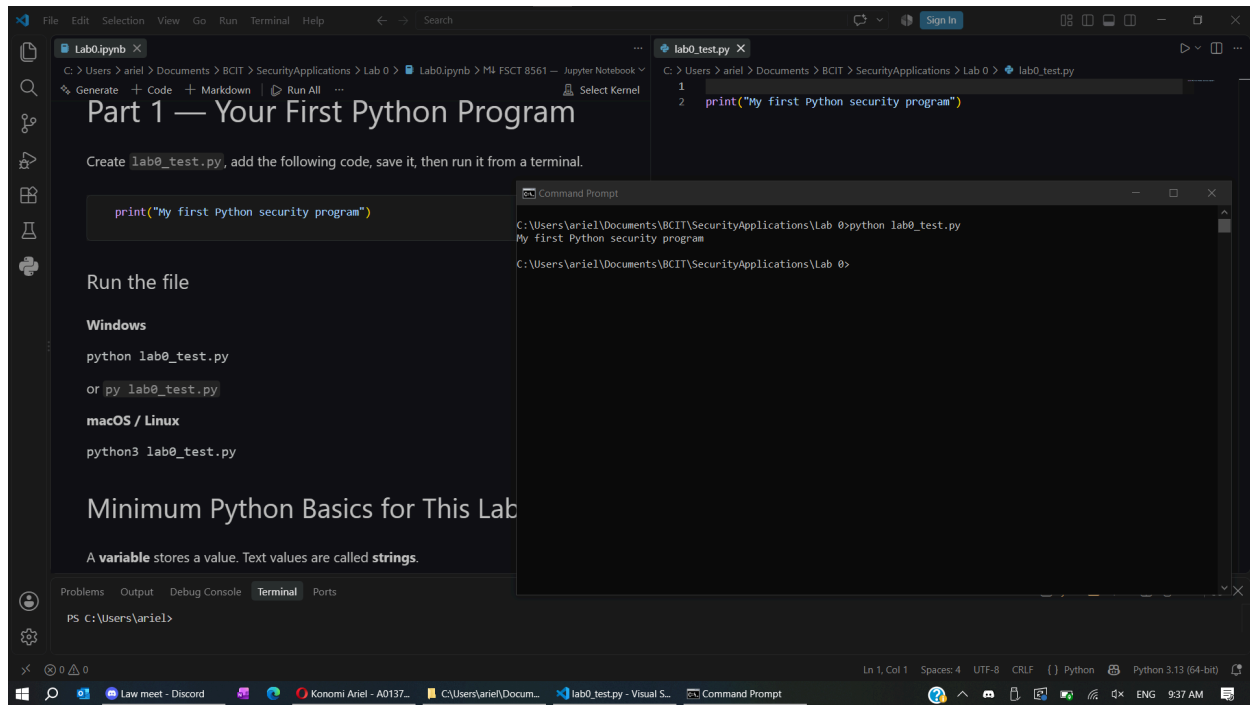
Lab 0: Introduction to Network Protocols Using Python Sockets

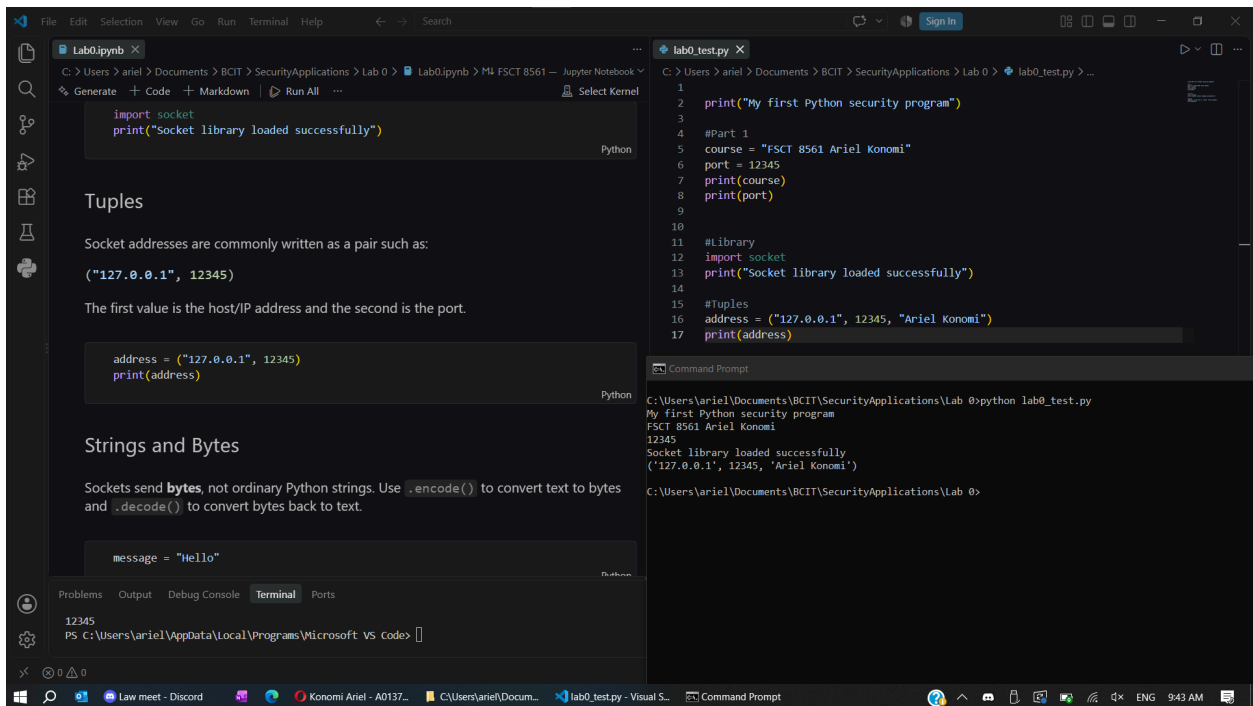
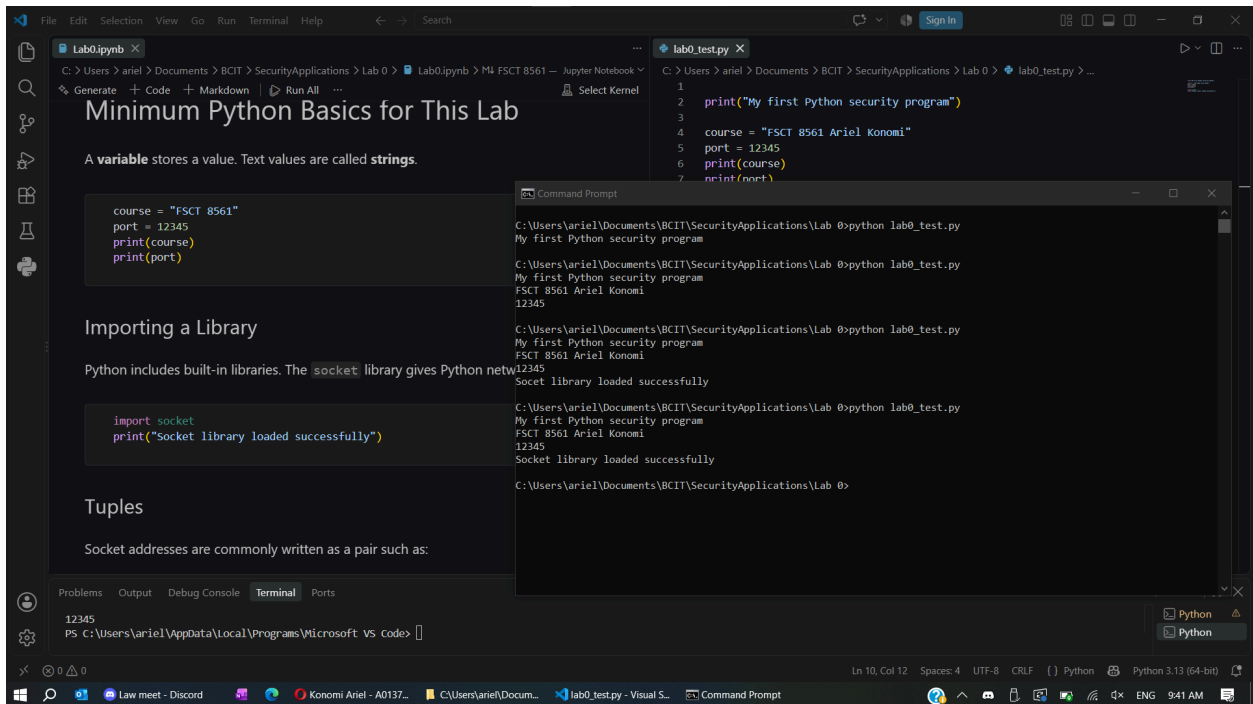
Part 0

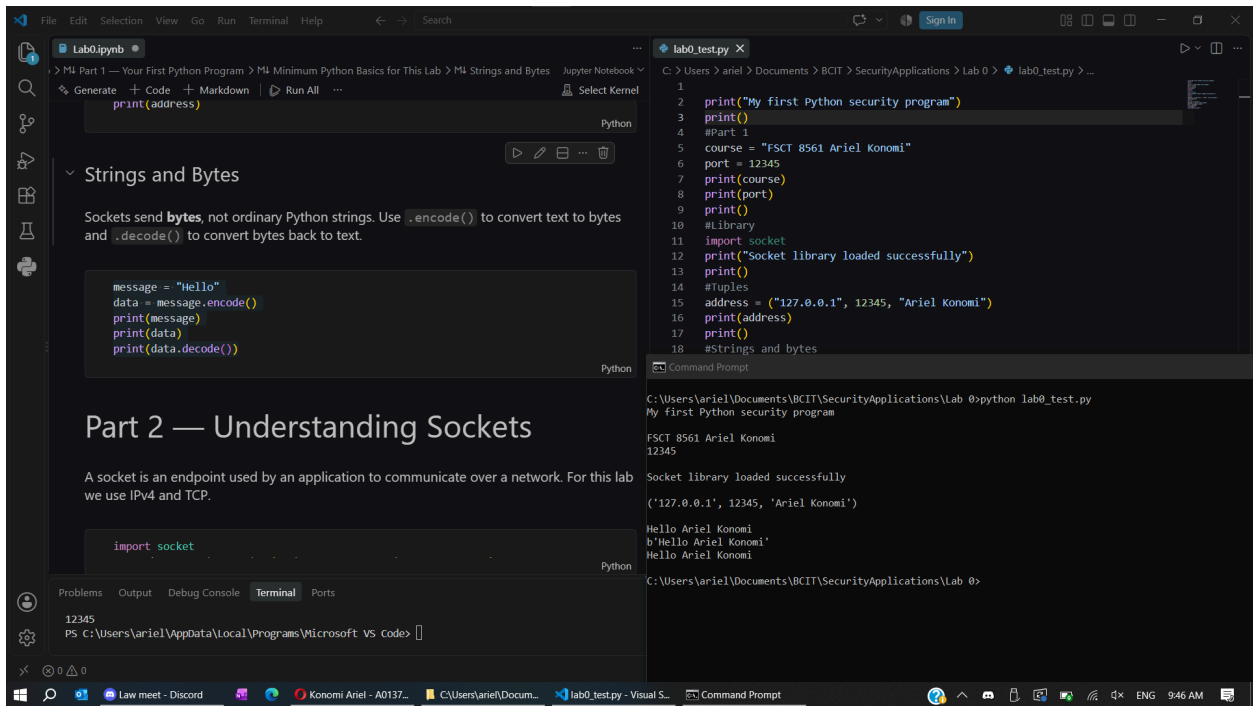


Part1

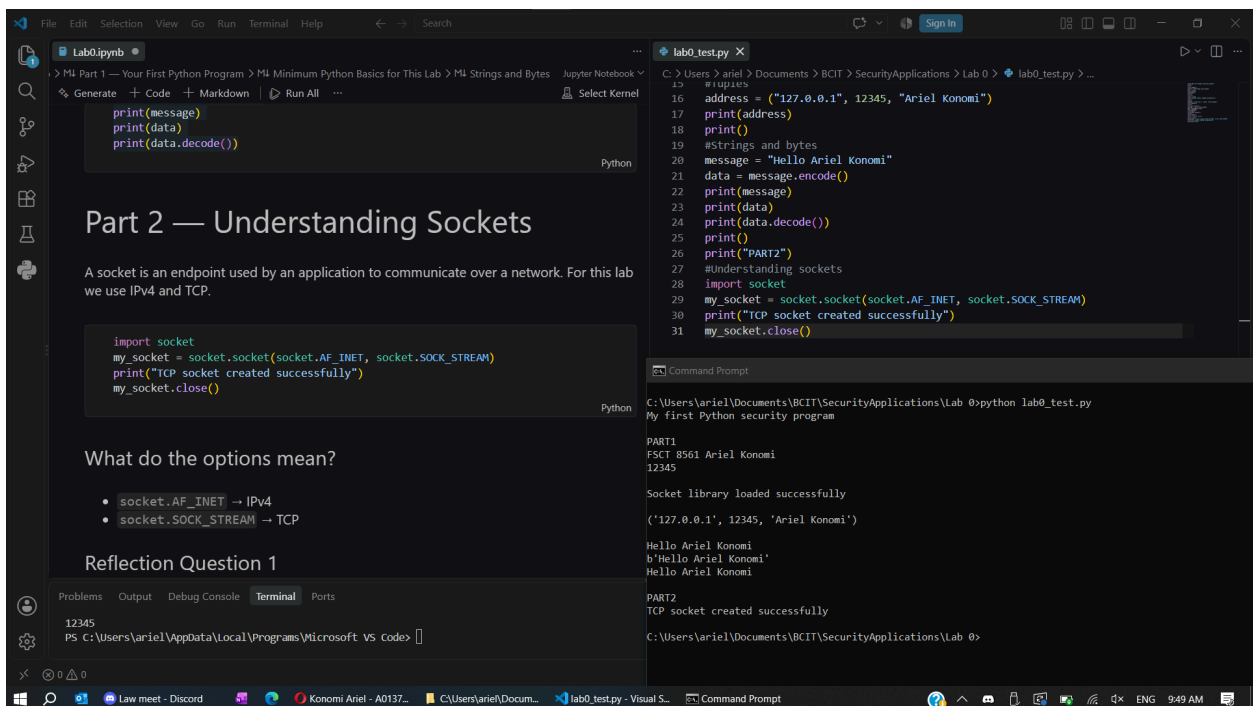
I ended up having to create a new directory for security applications and making sure that the names for the folders are easily accessible when trying to use cmd







Part2



Reflection Question 1

1. What is a socket?

A socket is an interface between the application and the network

2. What does `AF\_INET` mean?

It is an address family that specifies for the socket to use an IPv4 address

3. What does `SOCK\_STREAM` mean?

It's a TCP socket which gives reliable delivery. It guarantees in order delivery. It is also bidirectional

Part3

Server flow

socket() » bind() » listen() » accept() » recv() » send() » close()

Client flow

socket() » connect() » send() » recv() » close()

Start the server before the client.

Part4

The screenshot shows a Visual Studio Code editor with a file named `server.py` open. The code is a Python socket server that listens on port 12345 and responds to incoming connections with the message "Message received". The terminal at the bottom shows the command `python server.py` being executed, and the output "Server is waiting for a connection..." is displayed.

```
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server_socket.bind((HOST, PORT))
server_socket.listen(1)

print("Server is waiting for a connection...")

client_socket, client_address = server_socket.accept()

print("Connected by:", client_address)

data = client_socket.recv(1024)

message = data.decode()

print("Client says:", message)

reply = "Message received"

client_socket.send(reply.encode())

client_socket.close()
server_socket.close()
```

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
```

Part5

The screenshot shows a Visual Studio Code editor with a file named `client.py` open. The code is a Python socket client that connects to the server at 127.0.0.1 on port 12345 and sends the message "Hello Server". The terminal at the bottom shows the command `python client.py` being executed, and the output "Client says: Hello Server" is displayed.

```
import socket

HOST = "127.0.0.1"
PORT = 12345

client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

client_socket.connect((HOST, PORT))

message = "Hello Server"

client_socket.send(message.encode())

response = client_socket.recv(1024)

print("Server replied:", response.decode())

client_socket.close()
```

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Client says: Hello Server
```

Part6

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariel>cd "Documents\BCIT\SecurityApplications\Lab 0"

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server replied: Message received

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server replied: This is Lab0 for Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>
```

Change the server reply.

Task C — Small Challenge

Modify the server so it replies with:

Server received: <the client message>

```
server.py
client.py
```

```
el > Documents > BCIT > SecurityApplications > Lab 0 > server.py > ...
= "127.0.0.1"
= 12345

r_socket = socket.socket(
socket.AF_INET,
socket.SOCK_STREAM
```

```
Command Prompt

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 59291)
Client says: Hello Server

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 59346)
Client says: Hello Server, Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>
```

SyntaxError: invalid character '+' (U+002B)

PS C:\Users\ariel\AppData\Local\Programs\Microsoft VS Code >

Task C - Mini Challenge

```
Command Prompt

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:Hello Server, Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>
```

Part 7 — Connect to a Public Web Server

After your localhost client/server works, create `http_client.py`. This connects `example.com` on port 80 and sends a basic HTTP GET request.

```
server.py
client.py
http_client.py
```

```
> Documents > BCIT > SecurityApplications > Lab 0 > server.py > ...
("Connected by:", client_address)
= client_socket.recv(1024)
ge = data.decode()
("Client says:" message)
```

```
Command Prompt

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 60646)
Client says: Hello Server, Ariel Konomi
Client disconnected from the server...

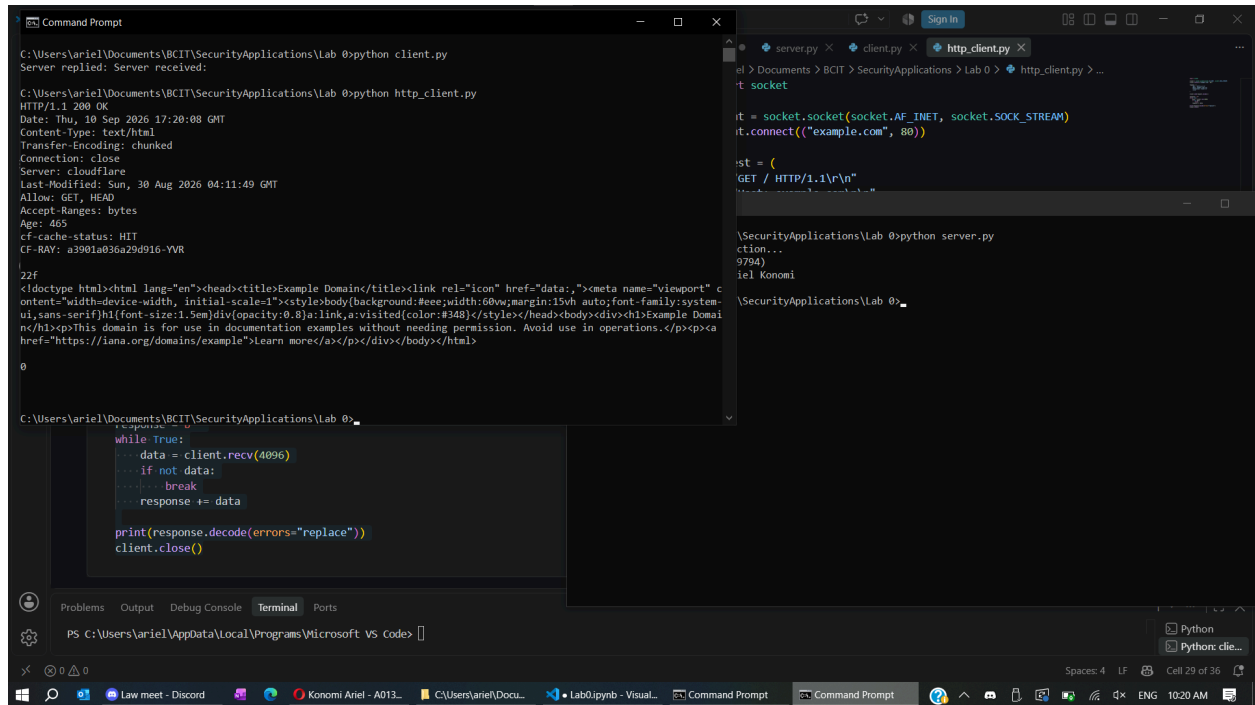
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 60675)
Client says: Hello Server, Ariel Konomi
Client disconnected from the server...

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 61439)
Client says: Hello Server, Ariel Konomi
Client disconnected from the server...

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>
```

PS C:\Users\ariel\AppData\Local\Programs\Microsoft VS Code >

Part7



```
Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server replied: Server received:

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python http_client.py
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 17:20:08 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Sun, 30 Aug 2026 04:11:49 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 465
cf-cache-status: HIT
CF-RAY: a3901a036a29d916-VVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" c
ontent="width=device-width, initial-scale=1"><style>body{background:#eee;width:600px;margin:15px auto;font-family:syste
ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domai
n</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a
href="https://iana.org/domains/example">Learn more</a></p></div></body></html>

0

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>

while True:
    data = client.recv(4096)
    if not data:
        break
    response += data

print(response.decode(errors="replace"))
client.close()
```

```
server.py
client.py
http_client.py
...
import socket

it = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
it.connect(("example.com", 80))

st = (
    GET / HTTP/1.1\r\n
    Host: example.com\r\n
    \r\n
)

\SecurityApplications\Lab 0>python server.py
ction...
9794)
iel Konomi

\SecurityApplications\Lab 0>
```

Reflection Question 2

1. Which host did the client connect to?

The host connected to example.com

2. Which port was used?

Port 80 was used

3. Why is port 80 relevant?

It is relevant as that is the http port, which is unencrypted

4. What did the server send back?

It sent back an http response, both the header and body of it

Part8

The screenshot shows a VS Code editor with a terminal window. The terminal output shows a simple socket server and client interaction. The server is running on port 1024 and the client is running on port 1024. The client sends a message "Hello Server, Ariel Konomi" and the server responds with "Hello Server, Ariel Konomi".

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client.py
Server received:Hello Server, Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client_socketio.py
Server received:Hello Server. This is for Part 8 of Lab0 - Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>
```

Answer in your report:

1. What could happen if a server accepts connections from anyone?
2. Is the Lab 0 message encrypted?
3. Could network traffic be observed by another system?
4. Why should servers validate input?
5. Why might authentication be useful?
6. Suggest two ways to make this application more secure.

The screenshot shows a VS Code editor with a terminal window. The terminal output shows a more complex socket server and client interaction using the socketio library. The server is running on port 8080 and the client is running on port 8080. The client sends a message "Hello Server, Ariel Konomi" and the server responds with "Hello Server, Ariel Konomi".

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python client_socketio-ari.py
Connection established by Ariel Konomi
Ariel disconnected from server

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 0>python server_socketio-ari.py
Client disconnected from the server...
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
Socket ID: 34Ay0hs0fhiegn0sAAAB
Data: {'data': 'my_data'}
```

```
28
29 app.router.add_get('/', index)
30
31 if __name__ == '__main__':
32     web.run_app(app)
33
34
```

Reflection Questions 3

1. What could happen if a server accepts connections from anyone?

A server could be easily overwhelmed by all of the requests where it could exhaust all of its memory and CPU. It could also become vulnerable to malicious attacks like DoS

2. Is the Lab o message encrypted?

The lab o message is unencrypted as it goes through the http port

3. Could network traffic be observed by another system?

It would be possible to be observed by another system as it is unencrypted

4. Why should servers validate input?

They should validate input mostly as a security check. They need to ensure that data and connections are secure

5. Why might authentication be useful?

Authentication is useful to verify who the client trying to connect is. Again, another step to ensure that connections are secure and trusted as well as accessed by the correct users

6. Suggest two ways to make this application more secure.

- Use port 443 instead of port 80 as data would be encrypted going through
- Require authentication before the connection is fully established

In this lab I learned a few of the basics of Python as well as how to set up a Github repository. This is my first time creating a repository within Github so it was quite interesting to learn it to set up for this course. It was also fun to be able to try and create a server and client from scratch and to put commands that need requirements to be able to function within the code